

MarginFi Audit Report

Table of Contents

1. Executive Summary

About MarginFi

Audit Summary

Overall Security Posture

Risk Profile

Launch Recommendations

Audit Scope

2. Assumptions and Considerations

Audit Assumptions

Centralization Assumptions

3. Severity Definitions

Impact

Likelihood

Severity Classification Matrix

4. Findings

M01: withdraw_all inside a rebalance sandwich wipes an unrelated order tag

M02: Incomplete oracle pricing will undercharge the protocol after a borrower withdraws rate-diluting collateral

M03: Liquidators can avoid the premium fee through a liquidation write-off

M04: A stale Marinade frontend cache will misprice mSOL positions in Marginfi

L01: Per-movement destination-rate checks will route aggregate deposits suboptimally

L02: Ignoring deposit-induced rate dilution will route funds to suboptimal native banks

L03: Stale JupLend borrow-rate simulation will lead to rebalancing into suboptimal venues

L04: start_rebalance fails when an unrelated balance has no side

L05: settle_rebalance_tip fails after the marginfi account is closed

L06: admin_close_account does not require an empty rebalance fee pool32

L07: Zero-weight collateral does not help health but still cuts the premium rate34

L08: Missing PT start-price checks will let borrowers create bad debt in same-asset e-mode
markets36

L09: Cached Exponent fields can overvalue impaired PT collateral38

L10: Marinade errors are not tagged as oracle errors40

L11: MAX_LST_SOL_RATE of 200 is not an economic cap42

L12: PTFixed treats hyUSD as \$144

5. Enhancement Opportunities46

E01: Recording the rebalance completion timestamp will improve health-cache visibility46

About Us47

About Adevar Labs47

Audit Methodology47

Confidentiality Notice49

Legal Disclaimer49

1. Executive Summary

About MarginFi

MarginFi is a decentralized lending and borrowing protocol on Solana. Users lend and borrow against a broad set of assets, including SOL, USDC, USDT, natively staked SOL, and Liquid Staking Tokens (LSTs), as well as positions held on integrated third-party venues such as Kamino and JupLend. Assets are organized into banks grouped under a shared administrator, and each user account can hold multiple collateral or liability positions, with initial and maintenance weights enforced at borrow and liquidation time.

This audit's scope centers on three recent feature additions:

- **Persistent auto-rebalance orders (PR 603):** A permissionless keeper flow that automatically migrates a user's deposits to the highest-yield bank for a given asset. Users configure a persistent order naming an allowlist of banks, a per-execution budget, and an optional keeper tip; any keeper can then execute a bounded, atomically-verified move once a yield spread justifies it.
- **Variable borrow premium (PR 615):** A per-user interest surcharge layered on top of base borrow interest, priced through a sparse, USD-weighted collateral/liability tag matrix. The premium accrues and settles alongside normal repayment and liquidation flows without requiring account resizing.
- **New oracle integrations (PR 631):** Dedicated pricing setups for mSOL (PythMSOL), other LSTs (PythLST), and Principal Tokens (PTPyth, PTFixed), extending price discovery to staked and yield-bearing collateral across the Kamino and JupLend integrations.

Audit Summary

The table below summarizes identified vulnerabilities by severity and remediation status.

Severity	Count	Fixed	Partially Fixed	Acknowledged
Critical	0	0	0	0
High	0	0	0	0
Medium	4	4	0	0
Low	12	10	0	2

Enhancement opportunities: 1, implemented.

Overall Security Posture

The MarginFi team demonstrated careful engineering around a complex rebalancing and premium-fee system, with no Critical or High severity issues identified. Findings instead reflect subtle edge cases in rate comparison, order-tag tracking, and oracle-input validation: areas that demand precise invariants under adversarial keeper and liquidator behavior. The team was quick to respond and often shipped fixes within days of a report.

This section describes the security posture before and after the fix review, the stage of the audit where Adevar Labs verified the fixes implemented by the MarginFi team.

After Fix Review Summary

The fix review confirmed that all 4 Medium and 10 of the 12 Low severity issues are fixed, and the enhancement is implemented. The order-tag wipe, both premium avoidance paths and the stale mSOL price are all closed.

2 Low severity issues are acknowledged. The team will likely price hyUSD PTs through PTPyth rather than the PTFixed \$1 approximation (L12), and a complete fix for the cached Exponent vault fields (L09) would require more accounts than current transactions can carry.

Before Fix Review Summary

The audit identified 4 Medium and 12 Low severity issues, along with 1 enhancement opportunity. The Medium findings center on the rebalance system's handling of order tags across withdraw/deposit sandwiches, the premium engine's exposure to oracle failures and liquidation write-offs, and a stale Marinade price cache affecting mSOL valuations. The Low findings are concentrated in the rebalance rate-selection logic (destination ranking, JupLend rate staleness, deposit-induced rate dilution), several rebalance lifecycle edge cases (fee-pool draining on admin close, tip settlement after account closure, unrelated-balance handling), and a cluster of oracle and pricing hardening items (PT start-price consistency, LST rate bounds, Marinade error tagging). The single enhancement opportunity concerns recording a rebalance completion timestamp to improve health-cache freshness.

Risk Profile

I. Rebalance (PR 603)

Severity	Count	Fixed	Partially Fixed	Acknowledged
Critical	0	0	0	0
High	0	0	0	0
Medium	1	1	0	0
Low	6	6	0	0

II. Premium Fee (PR 615)

Severity	Count	Fixed	Partially Fixed	Acknowledged
Critical	0	0	0	0
High	0	0	0	0
Medium	2	2	0	0
Low	1	1	0	0

III. Oracle & Price Feed Integrations (PR 631)

Severity	Count	Fixed	Partially Fixed	Acknowledged
Critical	0	0	0	0
High	0	0	0	0
Medium	1	1	0	0
Low	5	3	0	2

Launch Recommendations

All mandatory recommendations from the before-fix review are implemented. What remains:

I. Strongly recommended:

- Merge PR 673 and PR 686, which carry the last fixes, and deploy the reviewed code.
- Decide how PT positions should be liquidated while an Exponent vault is in emergency mode. With the new check in place, regular liquidations of these positions revert during that window. PR 685, which we have not reviewed, lets deleveraging proceed in this case.
- Have the changes listed as not reviewed in the Audit Scope audited before deploying them, in particular the Scope oracle integration (PR 661).

II. Post-Launch Monitoring:

- Monitor Exponent vaults backing PT collateral for emergency mode, and reconcile the cached `sy_for_pt` value against the live SY exchange rate (L09 is acknowledged).
- Monitor oracle adapter error rates for Marinade, Kamino, and JupLend integrations to catch stale or failing price feeds early.
- Watch premium rate snapshots that change after withdrawals or liquidations with unpriceable collateral, since the new ratchet can overcharge honest users until the next complete refresh.
- Monitor rebalance keeper activity for unusual sandwich patterns around stop-loss orders and the fee pool.

Audit Scope

- **Repository:** <https://github.com/0dotxyz/marginfi-v2>
- **Before Fix Review Commit Hash (PR 603):** `0bc3ab045524a880f7931e361a68d574f854cda0`
- **Before Fix Review Commit Hash (PR 615):** `d71ddf88a8db1fbcc51494cc9da2c074bfdb4e11`
- **Before Fix Review Commit Hash (PR 631):** `18d6fa421351c988856ad3ac3876de3cd65f216c`
- **After Fix Review Commit Hash (PR 603):** `63b41922747602d37506e6cb280d1b02bb85a248` (PR 673, not yet merged)
- **After Fix Review Commit Hashes (PR 615):** `ce6d14408e72db80ba637f70a3fb8871a76d966b` (PR 669, merged into `0.1.12-main` as `f8b0a37850ea74be16562409af9749e8ad357548`) and `37e1380d4da7d5ed7e567d9abaab4c00773c4b32` (PR 686, not yet merged)
- **After Fix Review Commit Hashes (PR 631):** `c0636e79cfa4ea81908105518fd6b22ccd1587eb` (PR 631) and `c2ff79441abed715008abdeafefbec4f447a4bda` (PR 661)
- **Files/Modules in Scope:** `programs/**/*.rs` (as modified by PRs)
- **Not reviewed:** the fix pull requests also carry changes that are not fixes for the findings in this report. These were not audited:
 - PR 631: `4b53c966` (make space for Scope), `8e02e07d` (renames in `price.rs` and `bank.rs`) and `fad12cfd` (reset `fixed_price` in `config_bank_oracle.rs`), plus lint and test-only commits
 - PR 661: the Scope oracle integration, except `c2ff7944`
 - PR 673: `507d6018` and `fa792dfa` (u32 conversions in `type-crate/src/types/interest_rate.rs`), `1d9c8d24` (JupLend struct layout) and `8ea6d43f` (JupLend rate model address check), plus the merge of `0.1.12-main` and lint commits
 - Any other pull request merged into the `0.1.11-main` or `0.1.12-main` branches

2. Assumptions and Considerations

Audit Assumptions

1. Deposit and repay do not refresh premium_rate_snapshot: You deposit SOL and borrow USDC at 0% premium, then deposit BONK (which has a premium against USDC) to keep health up. The USDC debt should then accrue the USD-weighted mix, including BONK. `lending_account_deposit` and `lending_account_repay` never call `update_premium_snapshots`: those ixes take no oracles, so they cannot reprice. The stored rate stays 0% until a later refresh (withdraw, borrow, `pulse_health`). A refresh always claims at the old rate first, so the stale window is never billed. The authors document this as an accepted approximation (PR description and module comments), and the team confirmed it is by design: deposit and repay must keep working when an oracle is down. We treat it as intended. Frontends should append `pulse_health` in the same transaction as the deposit.

2. Premium ratchet during oracle outages: Since the M02 fix, a withdrawal or a liquidator's own refresh that cannot price every collateral leg moves the premium rate up to the highest pair rate among the unpriced legs. During a genuine oracle outage this can overcharge an honest borrower until the next complete refresh. The team accepts this as the cost of keeping withdrawals and liquidations available.

3. Cached Exponent vault values: PT pricing relies on the `sy_for_pt` and `last_seen_sy_exchange_rate` values stored in the Exponent vault, which only update on operations that touch the vault (L09, acknowledged). We assume these values stay reasonably fresh and that PT banks are configured with conservative weights.

Centralization Assumptions

- A single group admin key configures every new oracle. A compromised or malicious admin can repoint any bank's oracle.
- The admin sets the PT `start_price`; the only bound is (0, 1] at set-time. No lower bound beyond >0, so a PT can be marked far below par or against a mismatched schedule.
- The admin selects and configures the premium fee rates. This is based on an off-chain risk calculation made by the protocol.

3. Severity Definitions

Each issue identified in this report is assigned a severity level based on two dimensions: **Impact** and **Likelihood**. These dimensions convey our team's understanding of both the potential consequences of a vulnerability and how likely it is to be discovered and exploited in the real world.

Note: Enhancements represent non-blocking improvements, typically usability, observability, or defense-in-depth tweaks, that do not pose an immediate asset risk but would improve the product's reliability and/or user experience if implemented.

Impact

Impact reflects the potential consequences of the issue, particularly on **project funds, user funds**, and the **availability or integrity** of the protocol.

- **High Impact:** Successful exploitation could result in a complete loss of user or protocol funds, disruption of core protocol functionality, or permanent loss of control over critical components.
- **Medium Impact:** Exploitation could cause significant disruption or partial loss of funds, but not a total compromise. May impact some users or non-core functionality.
- **Low Impact:** The issue has minor or negligible consequences. It may affect edge cases, expose metadata, or degrade performance slightly without putting funds or core logic at serious risk.

Likelihood

Likelihood reflects how easily an attacker can discover and exploit a vulnerability, as well as how economically attractive the exploit is.

- **High Likelihood:** The vulnerability is trivially exploitable. This means it can be exploited by a wide range of actors without privileged access rights, with minimal capital requirements and low financial risk.
- **Medium Likelihood:** This type of vulnerability can be found and exploited with moderate effort. It might require a significant capital investment, but the financial risk remains manageable.
- **Low Likelihood:** Exploitation of these vulnerabilities is often technically infeasible or requires highly specialized conditions. They may require extraordinary effort or entail significant financial risk for an attacker, with a high chance of failure and minimal potential return.

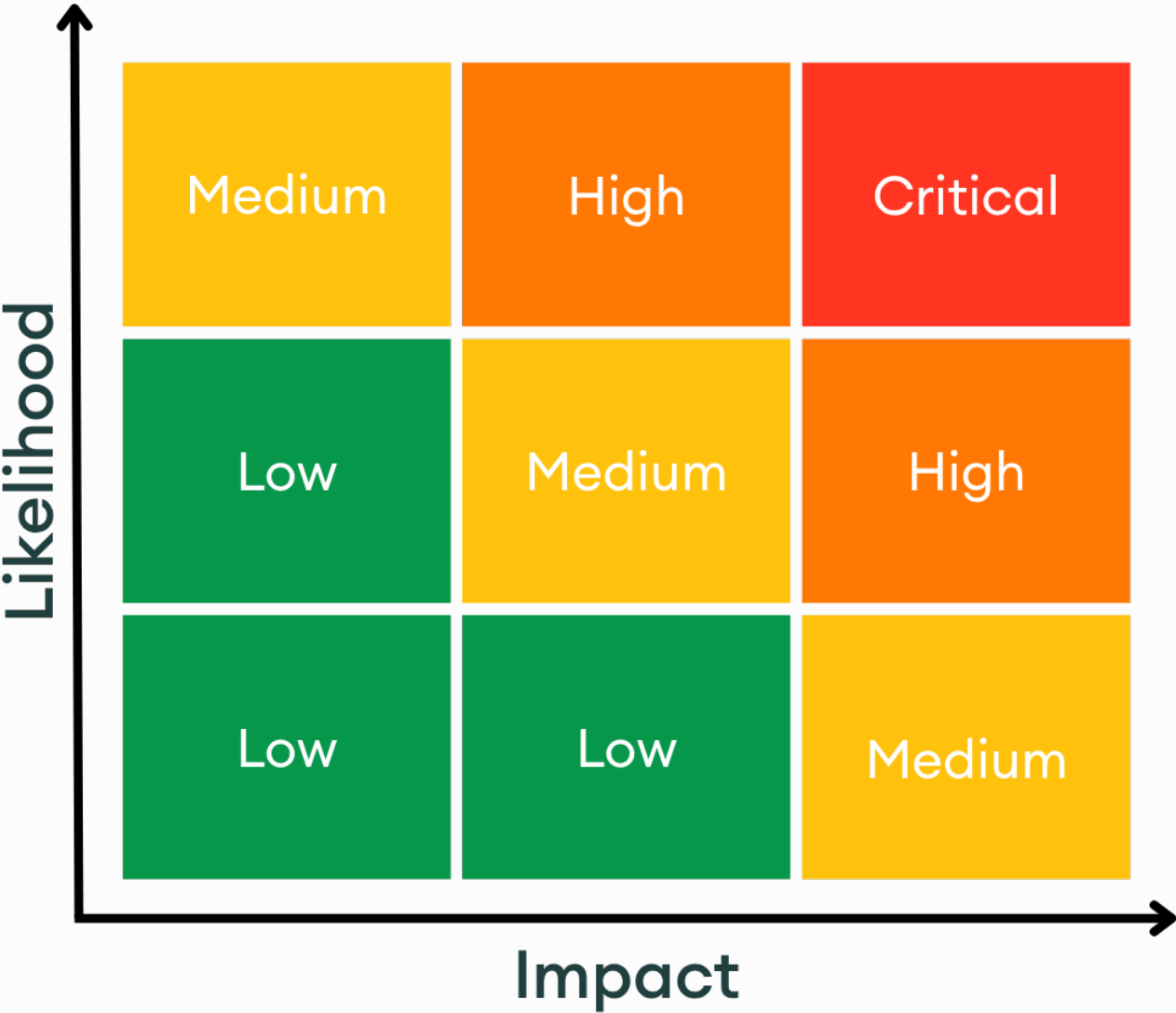
Severity Classification Matrix

By combining **Impact** and **Likelihood**, we assign a severity level using the matrix below:

- **Critical:** High impact with high likelihood (e.g., a bug that could allow anyone to drain a substantial amount of protocol funds with minimal effort)
- **High:** High impact with medium likelihood, or medium impact with high likelihood
- **Medium:** Medium impact and likelihood, or high impact/likelihood with low likelihood/impact

- **Low:** Low impact and likelihood, or medium impact/likelihood with low likelihood/impact

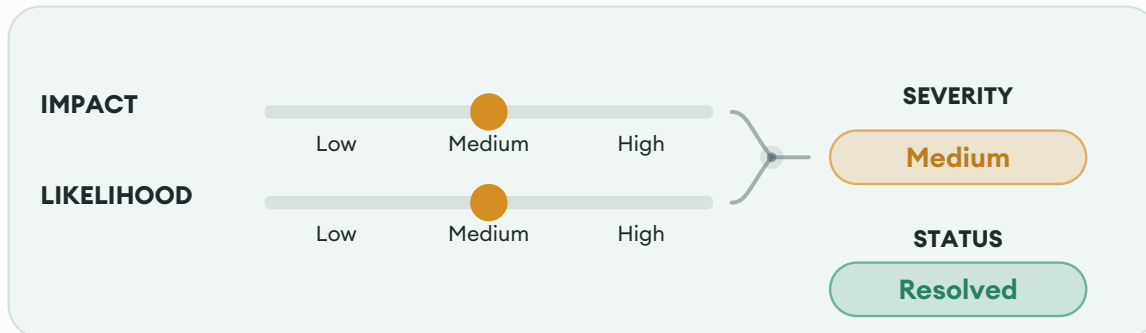
This structured approach helps teams prioritize fixes and mitigate the most dangerous threats first.



Severity Matrix

4. Findings

M01: **withdraw_all** inside a rebalance sandwich wipes an unrelated order tag



LOCATION

marginfi-v2/0bc3ab04/programs/marginfi/src/instructions/marginfi_account/withdraw.rs:L123 [↗](#)

RELEVANT SNIPPET

```

> _withdraw.rs RUST
121: let (amount_pre_fee, share_amount) = if withdraw_all {
122:     // Note: In liquidation, we still want this passed on the books
123:     bank_account.withdraw_all(in_receivership)?
124: } else {
125:     let amount_pre_fee = maybe_bank_mint
  
```

OTHER LOCATIONS

- [marginfi-v2/0bc3ab04/programs/marginfi/src/state/marginfi_account.rs:L2006](#) [↗](#)
- [marginfi-v2/0bc3ab04/programs/marginfi/src/state/marginfi_account.rs:L2119](#) [↗](#)
- [marginfi-v2/0bc3ab04/programs/marginfi/src/state/marginfi_account.rs:L1830](#) [↗](#)
- [marginfi-v2/0bc3ab04/type-crate/src/types/user_account.rs:L371](#) [↗](#)

DESCRIPTION

During a rebalance sandwich, deposit and withdraw pass **allow_rebalance = true**. **is_signer_authorized** then accepts any signer.

validate_rebalance_instructions binds each leg to the marginfi account. It does not bind the leg to an allowlisted bank.

A keeper may call **withdraw_all** on an unrelated tagged balance, then deposit the same tokens back.

withdraw_all closes the slot. **Balance::close** writes **empty_deactivated()**, which sets **tag** to 0.

The later deposit calls **find_or_create**. A new slot is created with **tag: 0**.

start_rebalance stores that unrelated slot's **tag** in **balance_states**. **verify_others_unchanged** compares bank, side, and shares. It never compares **tag**.

```
// programs/marginfi/src/state/rebalance.rs
slot.tag = balance.tag; // saved at start
// later, at end:
check!(I80F48::from(rec.shares) == I80F48::from(now), ...);
// rec.tag is never compared

// programs/marginfi/src/state/marginfi_account.rs
// find_or_create, new slot
tag: 0,
```

end_rebalance succeeds when shares restore exactly.

keeper_close_order closes an order when any saved tag is missing. After the wipe, anyone can close the stop-loss and keep the rent.

A partial withdraw does not clear the tag. Emptying an allowlisted source during an honest move is intended. That is not this bug.

Ordinary order execution snapshots **tag** the same way and also skips the comparison.

Example. Alice has USDC on the rebalance allowlist and SOL as stop-loss collateral. The SOL bank is idle. A keeper runs a valid USDC rebalance. In the same sandwich the keeper withdraws all SOL and deposits it back. SOL shares match. The SOL tag is 0. The keeper then closes Alice's stop-loss.

RECOMMENDATION

Compare **rec.tag** to **balance.tag** in **verify_others_unchanged**. Reject sandwich legs whose bank is not in the allowlist.

FIX REVIEW NOTES

Fixed in [34ecb373](#). Rebalance legs are now bound to the order's allowlist, and **carry_tags** verifies that no order tag is dropped during a rebalance.

M02: Incomplete oracle pricing will undercharge the protocol after a borrower withdraws rate-diluting collateral



LOCATION

marginfi-v2/d71ddf88/programs/marginfi/src/instructions/marginfi_account/withdraw.rs:L236-L262 [↗](#)

RELEVANT SNIPPET

```
>_withdraw.rs RUST

234: // Uses heap-efficient health check to support accounts with up to 16 positions
235: let group = marginfi_group_loader.load()?;
236: let mut premium_scratch = PremiumScratch::default();
237: check_account_init_health(
238:     &marginfi_account,
239:     &group,
240:     ctx.remaining_accounts,
241:     &mut Some(&mut health_cache),
242:     &mut Some(&mut premium_scratch),
243: );
244: health_cache.program_version = PROGRAM_VERSION;
245:
246: health_cache.set_engine_ok(true);
247: marginfi_account.health_cache = health_cache;
248:
249: // Inline CB gate: a risk-carrying withdraw (account carries a liability) reverts
   if
250: // any involved bank's live price has jumped past the breach threshold. A
251: // liability-free withdraw is risk-free and skips the gate.
252: if marginfi_account.lending_account.has_liabilities() {
253:     run_cb_price_gate(&marginfi_account, ctx.remaining_accounts)?;
254: }
255:
256: // Claim premium at the old rates and refresh every liability's premium rate snaps
   hot
257: // with the post-withdraw collateral mix.
258: marginfi_account.update_premium_snapshots(
259:     &group,
260:     &premium_scratch,
261:     clock.unix_timestamp as u64,
262: )?;
263: }
264:
```

DESCRIPTION

The premium engine calculates each liability's rate from the account's collateral as:

```
rate = weighted_sum / total_value, where:  
  weighted_sum = sum of (value * pair_rate) over collateral with value > 0 and pair_rate != 0  
  total_value  = sum of value                over ALL collateral entries
```

Collateral with a zero pair rate reduces the weighted rate. Removing that collateral can therefore increase the correct rate substantially. During an initial-health check, the health loop builds an oracle adapter for each active balance. If an asset adapter returns any error, the asset calculation records the error and assigns that asset zero value. This condition does not only match stale prices. It matches every adapter error, including **WrongOracleAccountKeys** from a caller-supplied oracle account.

The failed asset also sets **unpriceable_leg**, which leaves the premium scratch incomplete. This treatment is conservative for health because it removes collateral value. However, it prevents the premium engine from calculating the new collateral-weighted rate.

The borrow instruction rejects an incomplete premium pass through **refresh_unavailable()**. The withdraw instructions (including kamino/juplend/solend/drift variants) do not perform this check.

The stale rate persists until a complete premium refresh succeeds. The borrower can avoid account interactions, although a permissionless keeper can submit a health pulse. The impact depends on how quickly the keeper acts. A later successful refresh does not recover the missing premium. The update first claims the elapsed period at the old snapshot. It applies the correct rate only to future time. The protocol permanently loses the difference for the stale period. The loss increases with the borrowed amount, rate difference, and refresh delay.

Health calculations also project pending premium from the stale snapshot. They therefore understate the borrower's liabilities during this period. This condition can delay liquidation or permit an action that the correct accrued liability would reject.

PROOF OF CONCEPT

The borrower can exploit this difference as follows:

1. The borrower deposits two collateral assets.
2. The first asset has a nonzero pair rate for the planned liability. The second asset has a zero pair rate for that liability.
3. The borrower opens a premium-active liability. The zero-rate collateral reduces the initial weighted premium snapshot.
4. The borrower submits a withdrawal of the zero-rate collateral.
5. The borrower supplies an invalid oracle account for the nonzero-rate collateral. This produces an adapter error for that asset.
6. The health calculation excludes the nonzero-rate collateral. The remaining zero-rate collateral can still provide enough value for health to pass.
7. The premium update receives an incomplete scratch and returns successfully without changing the liability snapshot.

This caller-induced path works when the affected collateral bank does not enable the circuit-breaker price gate. A genuine oracle failure causes the same result. During such a failure, no party can complete a refresh until the oracle recovers.

RECOMMENDATION

Although rejecting a withdrawal when `premium_scratch.refresh_unavailable()` is true is a valid fix, we noted developer comments stating a requirement that withdrawals remain available despite oracle failures.

As an alternative solution, we propose a fallback calculation in case the premium scratch cannot be computed (i.e., when `refresh_unavailable` is true). Set the rate to:

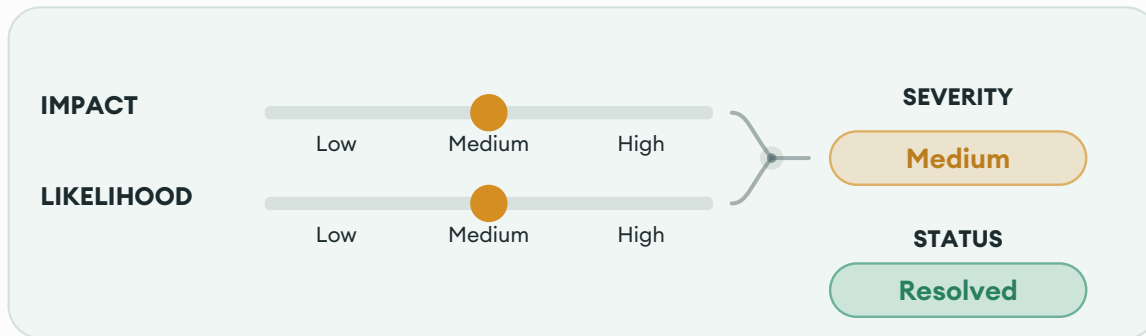
`max(prev_rate, known_collateral_rate, highest_unpriceable_rate)` where:

- `prev_rate`: the previous premium fee rate
- `known_collateral_rate`: the weighted average of all the collaterals that could be priced (i.e., no oracle error)
- `highest_unpriceable_rate`: the highest configured rate among all collaterals where there was an oracle failure

FIX REVIEW NOTES

Fixed in [96a8eec5](#) and [37e1380d](#). When the premium pass is incomplete, the withdraw paths (including the Kamino, JupLend, Solend and Drift variants) and the liquidator's own refresh in `check_liquidator_health_and_refresh_premium` now ratchet the snapshot to `max(previous rate, weighted rate of the priced collateral, highest pair rate among unpriceable legs)`, so it can only move up. The liquidatee's refresh does not ratchet, since a third party picks its oracles.

M03: Liquidators can avoid the premium fee through a liquidation write-off



LOCATION

marginfi-v2/d71ddf88/programs/marginfi/src/state/marginfi_account.rs:L2562-L2564 [↗](#)

RELEVANT SNIPPET

```
>_marginfi_account.rs RUST
2560: // legitimately forgiven). If full close ever becomes legal in liquidation, premium mu
    st
2561: // settle first – receivership's `repay_all` path is the model.
2562: if had_liabs && balance.is_empty(BalanceSide::Liabilities) {
2563:     balance.write_off_premium();
2564: }
2565:
2566: let share_amount = match operation_type {
```

DESCRIPTION

The protocol stores principal debt in **liability_shares** and variable-borrow premium separately in **premium_outstanding**. Normal repayment claims pending premium and settles that premium before it reduces principal. Liquidation does not use this settlement order when it credits seized collateral to the liquidator.

Let D be the liquidator's principal debt in **asset_bank**. Let P be the liquidator's accrued premium in that bank. Let A be the seized collateral amount.

An attacker can exploit the issue as follows:

1. The attacker borrows D from a premium-enabled bank and maintains sufficient collateral in other banks.
2. The attacker waits until the debt accrues a material premium P .
3. The attacker finds a liquidatable account that holds collateral in the same bank.
4. The attacker liquidates a different liability and selects an amount A that closes the attacker's principal debt.
5. The liquidation claims pending premium, which adds it to **premium_outstanding**.
6. The liquidation then applies all of A to principal because its calculation excludes **premium_outstanding**.
7. When the principal shares become empty, the function writes off the complete premium P .

The **ExhaustedLiability** condition does not prevent this result. Liquidation calls that condition with the liquidatee account and the selected **liab_bank**. It only ensures that the liquidatee retains debt in that

bank. The condition does not inspect the liquidator's debt in `asset_bank`. The two banks must be different, so the condition cannot cover the vulnerable balance.

The later initial-health check also cannot detect the loss. It evaluates the liquidator after the function `erased P`. The omitted premium can make an otherwise unhealthy liquidation pass the check.

The attacker consequently receives an unauthorized debt reduction or additional collateral. The protocol loses fee revenue and can accept a liquidator that does not satisfy the correct initial-health requirement. This result increases insolvency and bad-debt risk.

RECOMMENDATION

Settle seized collateral against the liquidator's premium before reducing principal.

First, claim the liquidator's pending premium in `asset_bank`. Then settle $\min(A, P)$ and credit that amount to `bank.collected_premium_outstanding`. Apply only the remaining amount to principal or a new asset position. This closely follows the repay flow.

Do not write off premium when a liquidation closes the liquidator's principal. Permit a write-off only for an explicit debt-forgiveness operation.

FIX REVIEW NOTES

Fixed in [4a4bcb77](#). Liquidation now settles the liquidator's outstanding premium before crediting the seized collateral to their account.

M04: A stale Marinade frontend cache will misprice mSOL positions in Marginfi



LOCATION

marginfi-v2/18d6fa42/programs/marginfi/src/state/lst_stake_price.rs:L96 [↗](#)

RELEVANT SNIPPET

```
> _lst_stake_price.rs RUST
94:  /// mSOL/SOL exchange rate = `msol_price / 2^32` (Marinade's cached rate).
95:  pub(crate) fn marinade_price_multiplier(state: &MinimalMarinadeState) -> MarginfiResul
    t<I80F48> {
96:      let rate = I80F48::from_num(state.msol_price())
97:          .checked_div(I80F48::from_num(MSOL_PRICE_PRECISION))
98:          .ok_or_else(math_error!())?;
```

DESCRIPTION

Marginfi calculates the mSOL price with this formula:

```
RUST
mSOL/USD = SOL/USD * (msol_price / 2^32)
```

Pyth supplies the SOL/USD price. Marinade's **State.msol_price** field supplies the mSOL/SOL rate.

Marinade defines **msol_price** as a frontend field. Its [source code](#) states that token calculations must not use this field.

```
> _mod.rs RUST
59:  pub available_reserve_balance: u64, // reserve_pda.lamports() - self.rent_exempt_for_t
    oken_acc.Virtual value (real may be > because of transfers into reserve). Use Updat
    e* to align
60:  pub msol_supply: u64, // Virtual value (may be < because of token burn). Use Update* t
    o align
61:  // For FE. Don't use it for token amount calculation
62:  pub msol_price: u64,
63:
```

Marinade calculates its canonical mSOL/SOL rate from its current accounting values:

RUST

```
total_virtual_staked_lamports / msol_supply
```

but it only updates **msol_price** with this formula during its stake-account crank process.

An attacker may observe this period of cache inconsistency and take advantage of the misprice.

RECOMMENDATION

Do not use **State.msol_price** for a Marginfi risk price. Calculate the mSOL/SOL rate from the same accounting values that Marinade uses:

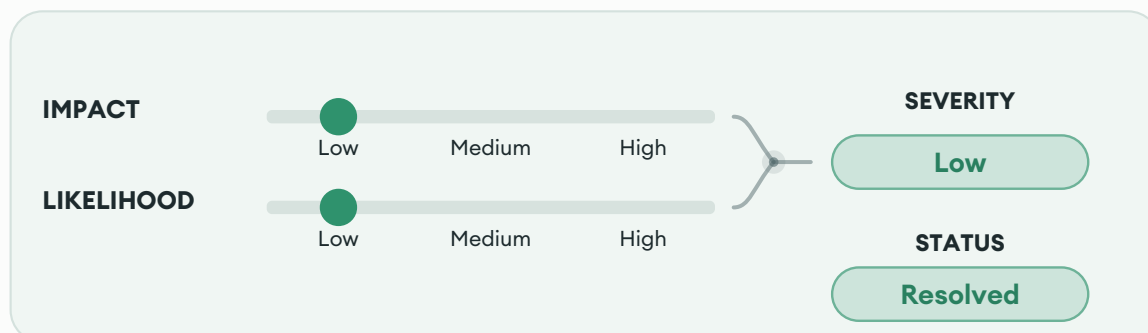
RUST

```
total_virtual_staked_lamports / msol_supply
```

FIX REVIEW NOTES

Fixed in [c0636e79](#). The mSOL rate is now computed as **total_virtual_staked_lamports / msol_supply** instead of reading the cached **msol_price**.

L01: Per-movement destination-rate checks will route aggregate deposits suboptimally



LOCATION

[marginfi-v2/0bc3ab04/programs/marginfi/src/instructions/marginfi_account/rebalance.rs:L853-L859](#)



RELEVANT SNIPPET

```
> _rebalance.rs RUST
851: };
852: // The destination must beat the source, as the source stands today, by the margin.
853: check!(
854:     dst_rate
855:     > rates[m.src_index as usize]
856:     .checked_add(min_imp)
857:     .ok_or_else(math_error!())?,
858:     MarginfiError::RebalanceNotImproving
859: );
860: // Banks this execution has already filled to their deposit capacity are skipped; no o
    ther
861: // bank may beat the destination at the same deposit amount.
```

OTHER LOCATIONS

- [marginfi-v2/0bc3ab04/programs/marginfi/src/instructions/marginfi_account/rebalance.rs:L876](#)

DESCRIPTION

start_rebalance validates each declared movement independently. When multiple movements deposit into the same destination bank, the rate calculation includes only the amount of the movement currently being checked. Consequently, every movement is evaluated against the destination's original state plus one movement, even though the destination will ultimately receive the sum of all movements.

The instruction already calculates total inflow per destination. However, that value is used only to determine whether a candidate bank has reached its deposit capacity; it is not passed into **rate_at**.

This allows a keeper to direct several individually valid movements into a relatively shallow, high-utilization bank. Each movement appears to select the best destination when considered separately, but their combined liquidity reduces the destination's utilization and supply rate enough that another allowed bank becomes superior.

If the selected destination continues to out-yield the source during the settlement window, the keeper can also receive the configured tip despite having produced a suboptimal allocation.

RECOMMENDATION

Aggregate all declared movements by destination before performing rate and capacity validation. The selected destination should then be evaluated using its complete planned inflow. For each movement, an alternative destination should be evaluated under the counterfactual state in which it receives both its existing planned inflow and the movement being tested. The validation should require the selected destination's rate under the complete submitted allocation to be at least as high as every feasible candidate's rate.

Deposit-capacity checks should also use aggregate amounts and ensure that the candidate has enough remaining capacity for the complete movement.

FIX REVIEW NOTES

Fixed in [89c789b2](#) and [42e4adf4](#). The destination rate is now computed over all planned deposits instead of one move at a time. A candidate bank with less room than the move is priced as if filled to its remaining capacity, so the comparison no longer overflows and reverts. [63b41922](#) also skips banks the move cannot legally enter.

L02: Ignoring deposit-induced rate dilution will route funds to suboptimal native banks



LOCATION

marginfi-v2/0bc3ab04/programs/marginfi/src/state/rate.rs:L64-L67 [↗](#)

RELEVANT SNIPPET

```
> _rate.rs RUST
62: ) -> MarginfiResult<I80F48> {
63:     let tag = bank.config.asset_tag;
64:     if matches!(tag, ASSET_TAG_DEFAULT | ASSET_TAG_SOL | ASSET_TAG_STAKED) {
65:         // Native banks price from the stored cache, so `extra_native` does not change
           the rate.
66:         return Ok(u32_to_milli(bank.cache.lending_rate));
67:     }
68:     let venue = venue.ok_or(MarginfiError::WrongNumberOfOracleAccounts)?;
69:     match tag {
```

DESCRIPTION

rate_at() is intended to calculate a bank's supply APR after receiving **extra_native** tokens. The external-integration implementations include these tokens when calculating post-deposit utilization. For native marginfi banks, however, **current_supply_apr()** ignores **extra_native** and returns the bank's current cached lending rate.

A deposit increases total supplied assets without increasing total liabilities. It therefore reduces utilization and, under the usual non-decreasing interest-rate curve, reduces the lender APR. Returning the pre-deposit rate causes native banks to appear more attractive than they will be after receiving the rebalance deposit.

RECOMMENDATION

Calculate native-bank APR using the hypothetical post-deposit utilization instead of returning the cached pre-deposit rate.

The implementation should:

1. Accrue the native bank before evaluating its rate, as currently done.
2. Calculate the bank's current total asset and liability amounts.
3. Add `extra_native` to total assets.
4. Calculate the resulting utilization.
5. Evaluate the bank's configured interest-rate curve using that utilization.
6. Return the resulting lender APR.

Do not store the hypothetical post-deposit asset amount in the bank cache. This should be a transient computation used only for comparing the APRs of the native banks.

FIX REVIEW NOTES

Fixed in [89c789b2](#). `current_supply_apr` now uses `NativeRateModel.rate_at` and accounts for `extra_native`.

L03: Stale **JupLend** borrow-rate simulation will lead to rebalancing into suboptimal venues



LOCATION

marginfi-v2/0bc3ab04/programs/juplend-mocks/src/state.rs:L306 [↗](#)

RELEVANT SNIPPET

```
>_ state.rs RUST
304: };
305: juplend_supply_rate_from_parts(
306:     u128::from(self.borrow_rate),
307:     u128::from(self.fee_on_interest),
308:     utilization,
```

DESCRIPTION

start_rebalance requires the selected destination to offer the highest post-deposit supply APR among the order's allowlisted banks. To perform this comparison, **rate_at** estimates each candidate's rate after receiving the proposed deposit.

For JupLend banks, this is calculated using the destination's reduced post-deposit **utilization** but continues using the reserve's stored pre-deposit **borrow_rate**.

```
RUST
let utilization = self.utilization_at(extra)?;

juplend_supply_rate_from_parts(
    u128::from(self.borrow_rate), // Pre-deposit rate
    u128::from(self.fee_on_interest),
    utilization,                  // Post-deposit utilization
    // ...
)
```

JupLend does not hold the borrow rate constant when liquidity changes. Its actual operation recalculates utilization after modifying the supply, evaluates the associated **RateModel**, and writes the resulting borrow rate into the reserve. Therefore, marginfi's preview combines state values that cannot coexist in the resulting JupLend reserve.

On a conventional increasing utilization curve, a deposit reduces both utilization and the borrow rate. The preview consequently overestimates the post-deposit supply APR:

```
marginfi preview = old borrow rate * post-deposit utilization * lender share
actual JupLend   = new borrow rate * post-deposit utilization * lender share
```

For a system that requires comparing exact APR values rather than upper bounds, this difference matters when selecting the best venue to rebalance into.

RECOMMENDATION

Reproduce JupLend's complete post-operation rate calculation instead of retaining the stale borrow rate. Use the [calc_borrow_rate_from_utilization](#) function and see the Jupiter source code [operate](#) function as an example.

> _rate_model.rs

RUST

```
82:
83: // Calculates borrow rate from utilization for a token
84: pub fn calc_borrow_rate_from_utilization(&self, utilization: u128) -> Result<u16> {
85:     let rate: u128 = if self.version == 1 {
86:         self.calc_rate_v1(utilization)?
87:     } else if self.version == 2 {
88:         self.calc_rate_v2(utilization)?
89:     } else {
90:         return Err(ErrorCodes::UnsupportedRateVersion.into());
91:     };
92:
93:     // Hard cap for borrow rate at maximum value 16 bits (65535)
94:     if rate > MAX_RATE.cast()? {
95:         emit!(LogBorrowRateCap { token: self.mint });
96:         return Ok(MAX_RATE);
97:     }
98:
99:     Ok(rate.cast())
100: }
101:
102: // Calculates the borrow rate based on utilization for rate data version 1 (with one k
    ink)
```

> _user.rs

RUST

```
144: // calculate utilization. If there is no supply, utilization must be 0 (avoid division
    by 0)
145: // Calculate utilization over scaled values to avoid precision loss from rounding.
146: let utilization: u16 = token_reserve.calc_utilization(
147:     supply_exchange_price,
148:     borrow_exchange_price,
149:     Some(supply_amount),
150:     Some(borrow_amount),
151: )?;
152:
153: // for borrow operations, ensure max utilization is not reached
154: if borrow_amount > 0 && utilization > token_reserve.max_utilization {
155:     return Err(ErrorCodes::MaxUtilizationReached.into());
156: }
```

>_user.rs

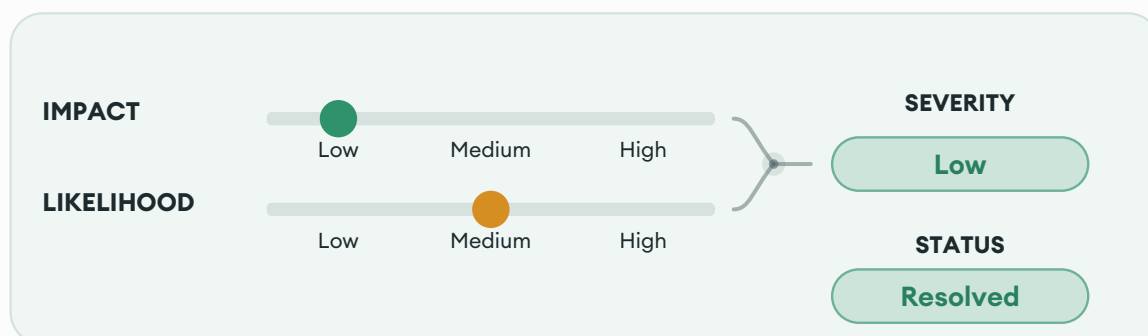
RUST

```
157:
158: let rate_model = ctx.accounts.rate_model.load()?;
159: let new_borrow_rate: u16 = rate_model.calc_borrow_rate_from_utilization(utilization.cast())?;
160:
161: // Update the TokenReserve account with new values
162: token_reserve.borrow_rate = new_borrow_rate;
163: token_reserve.last_utilization = utilization;
164: token_reserve.last_update_timestamp = Clock::get()?.unix_timestamp.cast()?;
165:
166: // update exchange prices
```

FIX REVIEW NOTES

Fixed in [a51e3aa3](#). The JupLend borrow rate is now computed at the post-deposit utilization using JupLend's own rate model.

L04: **start_rebalance** fails when an unrelated balance has no side



LOCATION

marginfi-v2/0bc3ab04/programs/marginfi/src/state/rebalance.rs:L183-L185 [↗](#)

RELEVANT SNIPPET

```
>_rebalance.rs RUST
181:     continue;
182: }
183: let side = balance
184:     .get_side()
185:     .ok_or(MarginfiError::IllegalBalanceState)?;
186: let slot = self
187:     .balance_states
```

DESCRIPTION

start_rebalance creates a **RebalanceRecord**. The record snapshots every active balance that is not on the allowlist.

For each of those balances it calls **get_side**. **get_side** returns **None** when both share counts are below **EMPTY_BALANCE_THRESHOLD** (1 native unit). The slot can stay **active** until the user calls **close_balance**.

initialize maps that **None** to **IllegalBalanceState**. The start instruction fails. The allowlisted move is not the cause.

```
// type-crate/src/types/user_account.rs
} else if I80F48::from(self.asset_shares) >= EMPTY_BALANCE_THRESHOLD {
    Some(BalanceSide::Assets)
} else {
    None
}
```

Example. Alice has a USDC rebalance order on banks A and B. She also has an active SOL slot with 0.5 native units. A keeper starts a valid A to B move. Ranking can pass. Record init then fails. The USDC path is valid. The SOL scrap is what stops start.

verify_others_unchanged later counts every active unreferenced slot. It does not use **get_side**. A skip in **initialize** only must match that end count, or end fails too.

RECOMMENDATION

Skip an active unreferenced balance when `get_side` returns `None`. Apply the same skip in `verify_others_unchanged`. Or return a dedicated error that tells the user to close the empty slot.

FIX REVIEW NOTES

Fixed in [6c98db35](#). Balances with no side are skipped when the `RebalanceRecord` is initialized.

L05: `settle_rebalance_tip` fails after the marginfi account is closed



LOCATION

marginfi-v2/0bc3ab04/programs/marginfi/src/instructions/marginfi_account/rebalance.rs:L1355-L1356



RELEVANT SNIPPET

```

>_rebalance.rs
1353: pub struct SettleRebalanceTip<'info> {
1354:     pub group: AccountLoader<'info, MarginfiGroup>,
1355:     #[account(has_one = group @ MarginfiError::InvalidGroup)]
1356:     pub marginfi_account: AccountLoader<'info, MarginfiAccount>,
1357:     #[account(
1358:         mut,
  
```

DESCRIPTION

`end_rebalance` moves the keeper tip out of the fee pool and into the `RebalanceRecord`.

The record PDA is `[REBALANCE_RECORD_SEED, marginfi_account, seq]`. `seq` has no bound. Close cannot list every record.

`close_account` requires empty balances, zero active orders, and an empty fee pool. It does not check for a live record.

The user may close the rebalance order while the record is still open. That is already allowed.

`close_rebalance_order` already accepts a closed marginfi account. It uses `UncheckedAccount`.

`SettleRebalanceTip` as shipped used `AccountLoader`. A closed account cannot load. Settlement then fails.

The escrowed tip and the record rent stay in the record. The keeper has no path to collect them.

```

// Original SettleRebalanceTip (PR 603 as shipped)
#[account(has_one = group)]
pub marginfi_account: AccountLoader<'info, MarginfiAccount>,

// close_rebalance_order already allows a closed account
/// CHECK: unchecked so the ix works even when the marginfi account was closed
pub marginfi_account: UncheckedAccount<'info>,
  
```

Example. A keeper finishes a tipped sandwich. The user closes the rebalance order. The user withdraws every deposit. The user drains the fee pool. The user closes the marginfi account. Anyone then calls

settle_rebalance_tip. The instruction fails. The keeper never receives the escrowed tip or the record rent.

RECOMMENDATION

Accept a closed marginfi account in **settle_rebalance_tip**. Use the same **UncheckedAccount** pattern as **close_rebalance_order**. Pay the recorded executor from the record. Do not require a live **AccountLoader**.

FIX REVIEW NOTES

Fixed in [4e47f10c](#). **settle_rebalance_tip** now takes the marginfi account as an **UncheckedAccount**, so the tip can be settled after the account is closed.

L06: **admin_close_account** does not require an empty rebalance fee pool



LOCATION

marginfi-v2/0bc3ab04/programs/marginfi/src/instructions/marginfi_account/admin_close.rs:L26-L39



RELEVANT SNIPPET

```

>_ admin_close.rs RUST

24: let is_inactive = elapsed > 60 * SECONDS_PER_DAY;
25:
26: check!(
27:     marginfi_account.can_be_closed() && is_inactive,
28:     MarginfiError::IllegalAction,
29:     "Account is not eligible for close (not empty or active within 60d)"
30: );
31:
32: check!(
33:     !marginfi_account.get_flag(ACCOUNT_IN_DELEVERAGE)
34:     && !marginfi_account.get_flag(ACCOUNT_IN_ORDER_EXECUTION)
35:     && marginfi_account.active_orders == 0
36:     && marginfi_account.liquidation_record == Pubkey::default(),
37:     MarginfiError::IllegalAction,
38:     "Account cannot be closed"
39: );
40:
41: emit!(AdminCloseAccountEvent {

```

OTHER LOCATIONS

- marginfi-v2/0bc3ab04/programs/marginfi/src/instructions/marginfi_account/close.rs:L32-L36 

DESCRIPTION

close_account rejects a close when the fee pool still holds lamports. The comment on that account says those lamports become unreachable if the marginfi account is closed.

admin_close_account is permissionless after 60 days of inactivity. It uses the same empty-balance check. As shipped, it did not load the fee pool. It did not require the pool to be empty.

withdraw_rebalance_fee_pool loads the marginfi account as **AccountLoader** and requires the account authority. After **admin_close_account** that account is closed. The withdraw instruction fails.

The fee pool PDA is **[REBALANCE_FEE_POOL_SEED, marginfi_account]**. It still holds the lamports.

A user recovers only if they initialize a new marginfi account at the same address and then withdraw. A typical PDA user opens the next account index. That new account has a different fee pool. The old lamports stay.

```
// programs/marginfi/src/instructions/marginfi_account/close.rs
check!(
    ctx.accounts.rebalance_fee_pool.lamports() == 0,
    MarginfiError::IllegalAction,
    "Withdraw rebalance fee pool before closing account"
);

// Original AdminCloseAccount (PR 603 as shipped)
// No fee-pool account. No lamports check.
```

Example. Alice sends 1 SOL to the fee pool. She withdraws every deposit. She does not withdraw the pool. After 61 days anyone calls **admin_close_account**. The 1 SOL stays in the old PDA.

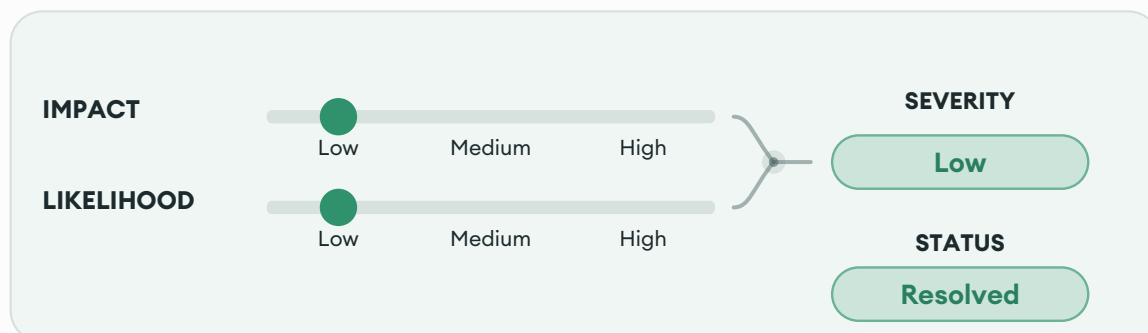
RECOMMENDATION

Add the fee pool account and the **lamports() == 0** check to **AdminCloseAccount**. Use the same guard as **close_account**.

FIX REVIEW NOTES

Fixed in [30c4670b](#). The rebalance fee pool must now be empty before the account is closed.

L07: Zero-weight collateral does not help health but still cuts the premium rate



LOCATION

[marginfi-v2/d71ddf88/programs/marginfi/src/state/marginfi_account.rs:L618-L628](#)

RELEVANT SNIPPET

```
>_marginfi_account.rs RUST
616:     balance_index: usize,
617: ) -> MarginfiResult {
618:     match balance.get_side() {
619:         Some(BalanceSide::Assets) => {
620:             let usd_value = if premium_price > I80F48::ZERO
621:                 && !matches!(bank.config.risk_tier, RiskTier::Isolated)
622:             {
623:                 calc_value(
624:                     bank.get_asset_amount(balance.asset_shares.into())?,
625:                     premium_price,
626:                     bank.get_balance_decimals(),
627:                     None,
628:                 )?
629:             } else {
630:                 I80F48::ZERO
```

OTHER LOCATIONS

- [marginfi-v2/d71ddf88/programs/marginfi/src/state/premium.rs:L345-L367](#)
- [marginfi-v2/d71ddf88/programs/marginfi/src/state/bank_config.rs:L50-L78](#)

DESCRIPTION

Account health values collateral with **asset_weight_init** and **asset_weight_maint**. A Collateral-tier bank may set both weights to 0. That collateral adds nothing to borrow power and nothing to liquidation health.

The variable-borrow premium mix does not use those weights. It averages pair rates by raw USD of each non-Isolated asset:

RUST

```
// programs/marginfi/src/state/marginfi_account.rs
calc_value(
    bank.get_asset_amount(balance.asset_shares.into())?,
    premium_price,
    bank.get_balance_decimals(),
    None,
)
```

Isolated banks also have weight 0. The mix forces Isolated legs to \$0, so they do not dilute the rate. Collateral-tier 0/0 banks are not Isolated. Their full USD enters the denominator on the next borrow, withdraw, or **pulse_health**. The stored premium rate falls even though health did not change.

A later liquidation seizes collateral that health actually counts. The 0/0-weight balance typically remains. The diluted snapshot remains with it.

A deposit with a real weight is different. That asset raises health, so a lower premium matches a safer account. A 0/0-weight asset does not raise health.

BankConfig::validate requires weight 0 only for Isolated. A Collateral bank may also be 0/0.

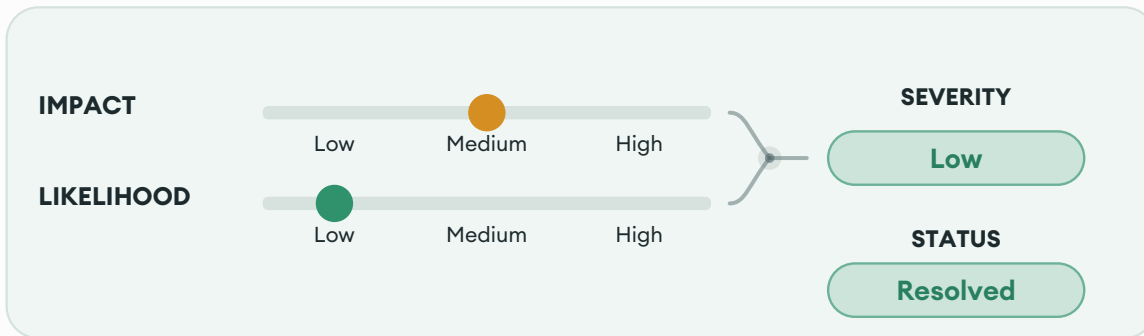
RECOMMENDATION

Use the same assets that contribute to health in the premium calculation. This means counting out both Isolated and 0/0-weight deposits.

FIX REVIEW NOTES

Fixed in [33c3cbbe](#). Collateral now counts toward the premium rate only when its **asset_weight_maint** is greater than zero.

L08: Missing PT start-price checks will let borrowers create bad debt in same-asset e-mode markets



LOCATION

marginfi-v2/18d6fa42/programs/marginfi/src/state/marginfi_account.rs:L923 [↗](#)

RELEVANT SNIPPET

```

>_marginfi_account.rs RUST

921:  /// key, or diverges from a previously seen mint/oracle-key/feed-family triple. Caller
922:  s must stop
923:  /// folding on `false`.
923:  fn update_reconciled_same_asset_config(
924:      shared_mint: &mut Option<Pubkey>,
925:      shared_oracle_key: &mut Option<Pubkey>,

```

DESCRIPTION

Same-asset e-mode gives a higher collateral weight when an account deposits and borrows the same token.

The **PTPyth** oracle calculates a principal token price as follows:

PT price = underlying Pyth price × PT multiplier

The PT multiplier moves from the bank's **fixed_price** to 1.0 during the vault's lifetime. The group administrator sets **fixed_price** independently for each bank. Therefore, two banks for the same PT mint can have different starting prices.

update_reconciled_same_asset_config verifies that the liability banks have the same:

- token mint
- primary oracle account
- oracle feed family

However, it does not verify that the banks have the same **fixed_price**. Consequently, two matching **PTPyth** banks can qualify for same-asset e-mode while reporting different prices for the same PT. This can allow a user to exploit the difference by using deposits made at the higher-price bank as collateral to borrow from the low-price bank and become under-collateralized as the PT approaches maturity.

RECOMMENDATION

Include **fixed_price** in the complete pricing identity used by same-asset e-mode.

Store the shared PT fixed price in **ReconciledSameAssetConfig**. When processing liability banks, require every **PTPyth** bank to have the same **fixed_price**.

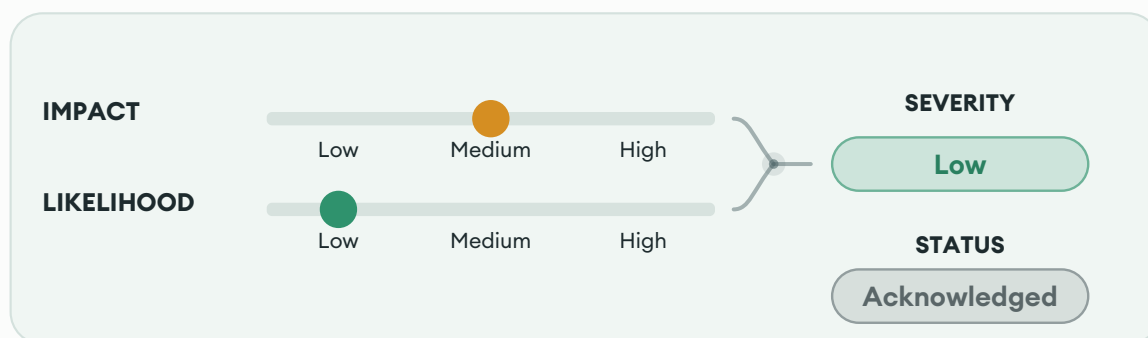
Before applying the enhanced collateral weight, require the collateral bank's **fixed_price** to match the reconciled value. Use the normal collateral weight if any comparison fails.

Also perform this validation when an administrator enables same-asset eligibility. This check will prevent the administrator from creating an inconsistent configuration.

FIX REVIEW NOTES

Fixed in [c0636e79](#). **fixed_price** is stored in **ReconciledSameAssetConfig** and checked in the collateral weight calculation, so a bank with a different start price falls back to the normal weight. The admin configuration path does not check it, but the runtime check is sufficient.

L09: Cached Exponent fields can overvalue impaired PT collateral



LOCATION

marginfi-v2/18d6fa42/programs/marginfi/src/state/lst_stake_price.rs:L265 [↗](#)

RELEVANT SNIPPET

```

> _lst_stake_price.rs RUST

263: );
264:
265: let sy_per_pt = I80F48::from_num(vault.sy_for_pt())
266:     .checked_div(I80F48::from_num(pt_supply))
267:     .ok_or_else(math_error!())?;

```

DESCRIPTION

The **sy_for_pt** field records the total SY amount that Exponent assigns to all outstanding PTs. The **sy_for_pt** is a stored snapshot that is not updated when the external SY exchange rate changes. It is only updated on an "operation that touches the PT supply or the last seen exchange rate" as per the Exponent comment [here](#).

```

> _vault.rs RUST

71:
72: /// The total SY set aside to back the PT holders
73: /// This value is updated on every operation that touches the PT supply or the last se
   en exchange rate
74: pub sy_for_pt: u64,
75:

```

Marginfi reads these stored values without a freshness check. Marginfi does not read the current SY state or check the current SY exchange rate. If there is an underlying loss in the SY token that is not captured by the Exponent vault, Marginfi does not know about it.

Although Exponent does not directly expose a freshness check mechanism, the current Marginfi dependency on the cached vault values alone is a risk.

RECOMMENDATION

Do not use Exponent's stored **sy_for_pt** and **last_seen_sy_exchange_rate** as the sole PT backing oracle. Check whether the Exponent vault is in an emergency mode before using the values.

FIX REVIEW NOTES

The team added the emergency mode check we recommended in [c2ff7944](#). On closer review, that check does not fully solve the issue. The emergency flag comes from the same cached vault state as **sy_for_pt**, so it is not fresher. It also makes liquidations revert while the vault is in emergency mode, because the oracle error is returned for the maintenance health check. A complete fix would read the SY program state through CPI, as Exponent does in [sy_cpi.rs](#).

DEVELOPER RESPONSE

Several of our transactions are already close to the account limit, and reading the SY state through CPI would require passing 5 additional account lists. We are leaving this as won't fix.

L10: Marinade errors are not tagged as oracle errors



LOCATION

marginfi-v2/18d6fa42/programs/marginfi/src/errors.rs:L804-L831 [↗](#)

RELEVANT SNIPPET

```

>_ errors.rs RUST

802:
803: impl MarginfiError {
804:     pub fn is_oracle_error(&self) -> bool {
805:         matches!(
806:             self,
807:             MarginfiError::WrongNumberOfOracleAccounts
808:             | MarginfiError::SwitchboardInvalidAccount
809:             | MarginfiError::PythPushInvalidAccount
810:             | MarginfiError::SwitchboardWrongAccountOwner
811:             | MarginfiError::PythPushInsufficientVerificationLevel
812:             | MarginfiError::WrongOracleAccountKeys
813:             | MarginfiError::PythPushStalePrice
814:             | MarginfiError::SwitchboardStalePrice
815:             | MarginfiError::StakePoolValidationFailed
816:             | MarginfiError::InvalidBankAccount
817:             | MarginfiError::MissingBankAccount
818:             | MarginfiError::MissingPythAccount
819:             | MarginfiError::MissingPythOrBankAccount
820:             | MarginfiError::PythPushInvalidWindowSize
821:             | MarginfiError::OracleMaxConfidenceExceeded
822:             | MarginfiError::ZeroSupplyInStakePool
823:             | MarginfiError::ExponentVaultValidationFailed
824:             // Lending protocol staleness errors - stale exchange rates mean unrel
825:             iable prices
826:             | MarginfiError::StakePoolStale // SPL / Sanctum stake pools
827:             | MarginfiError::ReserveStale // Kamino
828:             | MarginfiError::SolendReserveStale
829:             | MarginfiError::DriftSpotMarketStale
830:             | MarginfiError::JuplendLendingStale
831:         )
832:     }
833:     pub fn is_risk_engine_rejection(&self) -> bool {

```

OTHER LOCATIONS

- [marginfi-v2/18d6fa42/programs/marginfi/src/errors.rs:L277-L278](#) [↗](#)

DESCRIPTION

is_oracle_error does not include **MarinadeStateValidationFailed** (6136). It does include **StakePoolStale**, **StakePoolValidationFailed**, and **ExponentVaultValidationFailed**.

lending_account_pulse_health uses that list when init health returns an error. 6136 is not in the list, so pulse writes **oracle_ok = true**.

RUST

```
// programs/marginfi/src/instructions/marginfi_account/pulse_health.rs
if mfi_err.is_oracle_error() || health_cache.internal_err != 0 {
    health_cache.set_oracle_ok(false);
} else {
    health_cache.set_oracle_ok(true);
}
```

That can happen when Marinade load fails: wrong key, bad account, or a rate outside **(0, 200)**. Kamino mSOL and JupLend mSOL return the same error.

Init health returns the error for a failed liability and does not write **internal_err**. Pulse then has nothing else to flag. It still writes **engine_ok = false** and **mrngn_err = 6136**.

By contrast, the failed asset is treated as \$0 and stored in **internal_err**. Pulse then sets **oracle_ok** to false from that field.

Borrow and liquidate still fail the instruction. The protocol does not misprice the debt.

RECOMMENDATION

Add **MarinadeStateValidationFailed** to **is_oracle_error**.

FIX REVIEW NOTES

Fixed in [c2ff7944](#). **MarinadeStateValidationFailed** is now part of **is_oracle_error**, so Marinade load failures are reported as oracle errors.

L11: **MAX_LST_SOL_RATE** of 200 is not an economic cap



LOCATION

marginfi-v2/18d6fa42/programs/marginfi/src/state/lst_stake_price.rs:L21-L24 [↗](#)

RELEVANT SNIPPET

```
>_lst_stake_price.rs RUST
19: };
20:
21: /// Sanity ceiling for an LST/mSOL SOL-denominated exchange rate. Guards against a bad
    byte-slice
22: /// yielding an absurd multiplier: real rates sit near 1.0, so 200 leaves ample headro
    om while
23: /// still catching garbage (e.g. a misaligned read).
24: const MAX_LST_SOL_RATE: i128 = 200;
25: /// Upper bound (~5 years) on how far in the future a PT vault may mature, a sanity ch
    eck on the
26: /// timestamps decoded from the Exponent vault.
```

OTHER LOCATIONS

- [marginfi-v2/18d6fa42/programs/marginfi/src/state/lst_stake_price.rs:L95-L103](#) [↗](#)
- [marginfi-v2/18d6fa42/programs/marginfi/src/state/lst_stake_price.rs:L193-L199](#) [↗](#)

DESCRIPTION

LST and mSOL prices are Pyth SOL/USD times an on-chain SOL rate. The program rejects a rate that is not in **[0, 200]**.

That bound is a decode guard, not a risk cap. Live LST/SOL rates sit near 1.0–1.4. A wrong or hostile pool can still print 50x and pass. The load still checks SPL/Sanctum (or Marinade) owner and mint match. A normal user cannot swap the pool at price time. Admin listing still picks the account.

A 200x SOL mark would pass this check. Years of staking yield do not need that headroom.

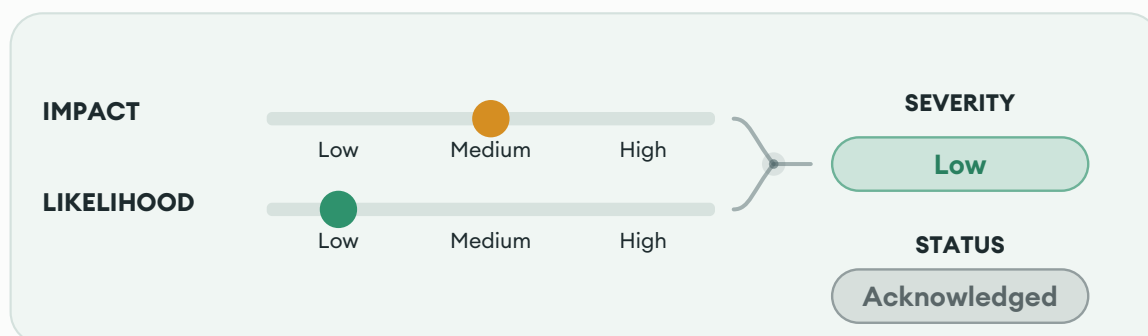
RECOMMENDATION

Lower **MAX_LST_SOL_RATE** to a tight ceiling such as 2 or 3. Treat listing of the pool or Marinade state as the real control. Do not treat 200 as a risk bound.

FIX REVIEW NOTES

Fixed in [c0636e79](#). **MAX_LST_SOL_RATE** is lowered from 200 to 3.

L12: PTFixed treats hyUSD as \$1



LOCATION

marginfi-v2/18d6fa42/programs/marginfi/src/state/price.rs:L1190-L1211 [↗](#)

RELEVANT SNIPPET

```

>_ price.rs RUST
1188:         })
1189:     }
1190:     OracleSetup::PTFixed => {
1191:         // (0) Exponent vault only. hyUSD ~= $1, so the PT linear rate is the USD price
1192:         // directly (a time-varying fixed feed), with no base oracle to multiply.
1193:         check!(ais.len() == 1, MarginfiError::WrongNumberOfOracleAccounts);
1194:
1195:         let vault_loader = load_exponent_vault(bank_config, &ais[0], 0)?;
1196:         let vault = vault_loader.load()?;
1197:         let start_price: I80F48 = bank.config.fixed_price.into();
1198:         let pt_price = pt_linear_multiplier(&vault, clock, start_price)?;
1199:
1200:         let feed = FixedPriceFeed { price: pt_price };
1201:         let cache_raw_price = if let Some(price_type) = cache_price_type {
1202:             Some(feed.get_price_and_confidence_of_type(price_type, u32::MAX)?)
1203:         } else {
1204:             None
1205:         };
1206:
1207:         Ok(OracleLoadContext {
1208:             adjusted_price_feed: OraclePriceFeedAdapter::Fixed(feed),
1209:             cache_raw_price,
1210:             cache_multiplier: I80F48::ONE,
1211:         })
1212:     }
1213: }
```

OTHER LOCATIONS

- marginfi-v2/18d6fa42/programs/marginfi/src/instructions/marginfi_group/set_oracle_price.rs:L22-L25 [↗](#)

DESCRIPTION

PTFixed prices a PT with no Pyth base feed. USD mark is the linear vault rate alone. The comment states the underlying is about \$1.

That is a listing assumption that does not hold at all times. CoinGecko for Hylo USD ([hyusd](#)) over the last year shows the peg is usually near \$1, but it does move. The largest dip in that window is about 7% (near \$0.93). Those soft patches last several days, not a single print.

That is not USDC-like. [PTFixed](#) cannot see it. Health still walks the line toward \$1. The vault cap is in underlying per PT, not in USD. The circuit breaker sees the time walk, not a depeg.

[PTFixed](#) is not hyUSD-only. Any PT listed this way gets the same \$1 base.

This is not an acceptable silent invariant for borrow collateral. It can be an acceptable listing choice only if LTV already assumes a depeg. [start_price](#) does not fix a later depeg. The line still ends at 1.0.

RECOMMENDATION

Do not give [PTFixed](#) material borrow power unless the underlying has a live USD feed. Prefer [PTPyth](#) when a Pyth USD feed exists. Until then, use a low asset weight, a tight deposit cap, and no e-mode lift.

DEVELOPER RESPONSE

We will likely not use this oracle setup for hyUSD and will instead set up a PTPyth bank with a hyUSD/USD oracle, but we keep [PTFixed](#) among the options. If we use it, the conservative asset weights mitigate the approximation. The code comment is clarified in [c0636e79](#).

5. Enhancement Opportunities

E01: Recording the rebalance completion timestamp will improve health-cache visibility

LOCATION

marginfi-v2/0bc3ab04/programs/marginfi/src/instructions/marginfi_account/rebalance.rs:L993 [↗](#)

RELEVANT SNIPPET

```
>_rebalance.rsRUST
991: }
992:
993: let mut health_cache = HealthCache::zeroed();
994: let (value_moved, tip_pending, move_yield_indices) = {
995:     let account = ctx.accounts.marginfi_account.load()?;
```

DESCRIPTION

end_rebalance creates a zero-initialized **HealthCache**, populates its maintenance-health values, and sets **PROGRAM_VERSION** and the engine-status flags. It then overwrites the account's existing cache with this new value. However, it does not set **health_cache.timestamp**. Consequently, every successful rebalance stores a timestamp of 0, even though the maintenance-health values were calculated using the current clock and account state.

The omission does not currently affect on-chain solvency enforcement because security-sensitive instructions recompute health rather than trusting this cache. It can nevertheless cause off-chain consumers to treat valid maintenance-health data as uninitialized or out of date.

POTENTIAL BENEFIT

Setting the timestamp would give accurate freshness metadata to health caches written by successful **end_rebalance** calls. It would eliminate the deterministic **timestamp == 0** state after rebalances and reduce unnecessary cache refreshes and temporary gaps in UI or monitoring data.

RECOMMENDATION

Set the timestamp immediately after zero-initializing the cache:

```
RUST
let mut health_cache = HealthCache::zeroed();
health_cache.timestamp = clock.unix_timestamp;
```

FIX REVIEW NOTES

Implemented in [24cae570](#). **end_rebalance** and **end_execute_order** now set the health cache timestamp.

About Us

About Adevar Labs

Adevar Labs is a boutique blockchain security firm specializing in Web3 audits.

The firm was built by a mix of experienced professionals from traditional enterprise security and crypto natives who have contributed to some of the most critical projects in blockchain infrastructure.

Our team's background spans companies like Bitdefender, Asymmetric Research, Quantstamp, Chainproof, and Juicebox, and includes experience securing smart contracts, bridges, and L1 and L2 protocols across ecosystems like Solana, Ethereum, Polkadot, Cosmos, and MultiversX.

With over 100 audits completed and a portfolio that includes custom fuzzers, exploit modeling, and runtime testing frameworks, Adevar Labs brings both depth and precision to every engagement.

Our auditors have discovered critical vulnerabilities, built high-impact tooling, and placed in top positions in premier audit competitions including Code4rena and Sherlock.

Team members hold distinctions such as PhDs in software protection, key roles at Fortune 500 companies, and leadership of flagship conferences like ETH Bucharest.

With team members whose publications have earned over 1,100 academic citations, and with the trust of projects securing over \$500M in on-chain value, Adevar Labs blends elite technical rigor with real-world security impact.

We also collaborate with some of the best independent security researchers in the Web3 space.

Projects may optionally request specific contributors to be part of their audit.

Audit Methodology

Our audit methodology is designed to provide thorough security assessments of Solana programs. We focus explicitly on rigorous manual analysis, detailed threat modeling, and careful validation of implemented fixes to ensure your programs operate securely and as intended.

1. Program Context and Architecture Analysis

Our auditors begin by deeply examining your Solana program's documentation, intended functionality, and account design. We meticulously map out program interactions, instruction processing flows, and state management logic. Special attention is given to understanding how your program interfaces with critical Solana system programs, such as the SPL Token Program, Stake Program, and System Program. Finally, we examine external integrations with other Solana projects and verify that inputs and return values are handled properly.

2. Threat Modeling

We conduct targeted threat modeling tailored specifically for Solana's execution environment. We carefully define attacker capabilities and identify potential vulnerabilities that may arise from Solana-specific issues, including but not limited to:

- Unauthorized account data manipulation
- Improper ownership or signer verification
- Misuse of Program-Derived Addresses (PDAs)
- Incorrect use of Cross-Program Invocations (CPI)
- Failure to adequately handle account privileges or account states
- Risks stemming from rent-exemption and account initialization logic

3. In-depth Manual Security Review

Our experienced auditors perform an extensive manual security review of your Rust-based Solana programs. This involves a comprehensive line-by-line inspection of source code, focusing on common Solana vulnerabilities including, but not limited to:

- Missing or insufficient ownership checks
- Inadequate signer checks
- Incorrect handling of CPI calls and invocation privileges
- Arithmetic and integer overflow or underflow errors
- Unsafe deserialization and serialization of account data structures
- Improper token transfers and SPL-token logic issues
- Logic flaws in financial operations or state transitions
- Edge-case handling in instruction input validation
- Potential denial-of-service vectors related to transaction execution and account handling

During this stage, we clearly document any discovered vulnerabilities, including detailed descriptions, precise severity ratings, and recommendations for secure implementation. In situations where it is not clear how the vulnerability might be exploited, we may also include a detailed proof-of-concept exploit with code snippets and instructions on how the attack could be performed.

4. Detailed Fix Review and Validation

After the initial audit and your team's subsequent remediation efforts, we perform a comprehensive fix review to ensure the vulnerabilities identified have been effectively resolved.

We verify each fix individually, confirming that:

- Corrections effectively eliminate the security risks
- Changes do not inadvertently introduce new vulnerabilities or regressions
- The fixes align closely with Solana best practices and secure coding guidelines

Our detailed validation ensures that security improvements are robust, complete, and aligned with best practices specific to the Solana development ecosystem.

Confidentiality Notice

This report, including its content, data, and underlying methodologies, is subject to the confidentiality and feedback provisions in your agreement with Adevar Labs. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Adevar Labs.

Legal Disclaimer

The review and this report are provided by Adevar Labs on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Adevar Labs disclaims all warranties, express or implied, in connection with this report, its content, and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

You agree that access to and/or use of the report and other results of the review, including any associated services, products, protocols, platforms, content, and materials, will be at your sole risk.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time of the engagement.

You acknowledge that blockchain technology remains under development and is subject to unknown risks and flaws. Adevar Labs does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open-source or third-party software, code, libraries, materials, or information accessible through the report. As with the purchase or use of a product or service in any environment, you should use your best judgment and exercise caution where appropriate.